

실습 준비

실행되는 개발 환경 만들기

프로젝트를 받고, 프로젝트 전용 Python을 VS Code에 연결해 첫 실행까지 확인합니다

일곱 단계를 따라가면 강사의 설명 없이 혼자 끝까지 완료할 수 있습니다.

다음 → 준비가 끝나면 어떤 화면이 보이는지 먼저 봅니다

이 화면이 보이면 준비가 끝난 것입니다

먼저 도착 지점을 봅니다. 앞으로의 모든 단계는 이 화면을 만들기 위한 것입니다.

① 프로젝트 폴더

VS Code 왼쪽에
ai-advanced-03 폴더가 열려 있습니다

② .venv 폴더

프로젝트 안에
.venv 폴더가 보입니다

③ 선택된 Python

상태 표시줄에
.venv\Scripts\python.exe 가 보입니다

④ 성공한 터미널

아래 터미널에서
확인 명령이 오류 없이 끝납니다

이 네 가지가 오늘의 최종 완료 증거입니다

47쪽 체크리스트에서 같은 네 가지를 다시 확인합니다.

완료 증거

네 가지가 한 화면에 보이면 개발 환경 구성이 끝난 것입니다

다음 → 실습 파일만 받아서는 왜 바로 실행되지 않을까요

파일만 받아서는 실행되지 않습니다

실습 파일은 글자일 뿐입니다. 글자를 실행할 도구가 함께 있어야 합니다.

지금 상태 — 파일만 있음

- 코드 파일은 있습니다.
- 실행할 Python이 없습니다.
- 코드가 쓰는 기능(패키지)이 없습니다.
- 파일을 열어 볼 작업 화면이 없습니다.



목표 상태 — 연결 완료

- 프로젝트 폴더가 열려 있습니다.
- 프로젝트 전용 Python이 선택되어 있습니다.
- 필요한 기능이 모두 설치되어 있습니다.
- 터미널에서 실행 결과를 볼 수 있습니다.

네 가지를 순서대로 연결하는 일이 개발 환경 구성입니다.

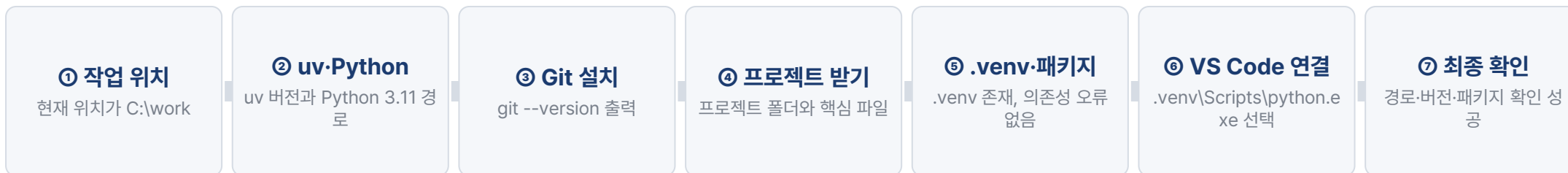
핵심

파일 + 실행 도구 + 필요한 기능 + 작업 화면이 연결되어야 실행됩니다

다음 → 그 연결을 일곱 단계로 나눕니다

준비 과정은 일곱 단계입니다

각 단계에는 끝났는지 알려 주는 완료 증거가 하나씩 있습니다.



회색 = 아직, 주황 = 지금 하는 단계, 초록 ✓ = 완료된 단계

일곱 번째 칸까지 초록으로 바뀌면 실습을 시작할 수 있습니다.

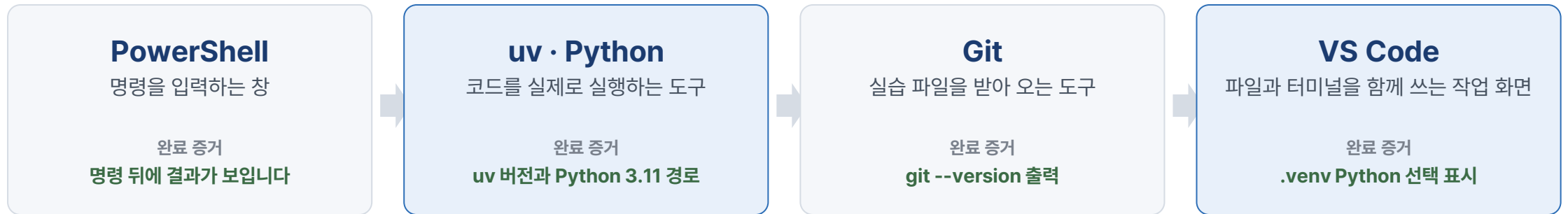
전체 지도

이 일곱 칸은 모든 구간에서 같은 위치·같은 색·같은 이름으로 다시 나옵니다

다음 → 네 도구가 각각 무슨 일을 하는지 봅니다

네 도구는 서로 다른 일을 맡습니다

도구 이름보다 각각이 무슨 일을 하는지 먼저 기억합니다.



용어를 미리 한 줄로

PATH = 어느 폴더에서도 명령을 찾게 해 주는 위치 목록 / 패키지 = 실습 코드가 사용하는 기능 묶음

핵심

명령을 넣는 창, 코드를 실행하는 도구, 파일을 받는 도구, 작업하는 화면

다음 → 시작하기 전에 여섯 가지를 확인합니다

시작하기 전에 여섯 가지를 확인합니다

하나라도 확인되지 않으면 설치를 시작하지 말고 교육 담당자에게 먼저 확인합니다.

① Windows 버전

Windows 10 또는 11 인지 확인합니다

② 64비트

설정 → 시스템 → 정보에서 64비트인지 확인합니다

③ 인터넷 연결

설치 파일을 내려받을 수 있어야 합니다

④ 디스크 여유

최소 10GB 이상 남아 있는지 확인합니다

⑤ 설치 권한

일반 권한으로 프로그램을 설치할 수 있는지 확인합니다

⑥ 저장소 주소

교육 담당자가 안내한 주소가 맞는지 확인합니다

설치 권한이나 보안 정책이 막힌다면 관리자 권한으로 다시 시도하지 말고 교육 담당자에게 확인합니다.

원칙

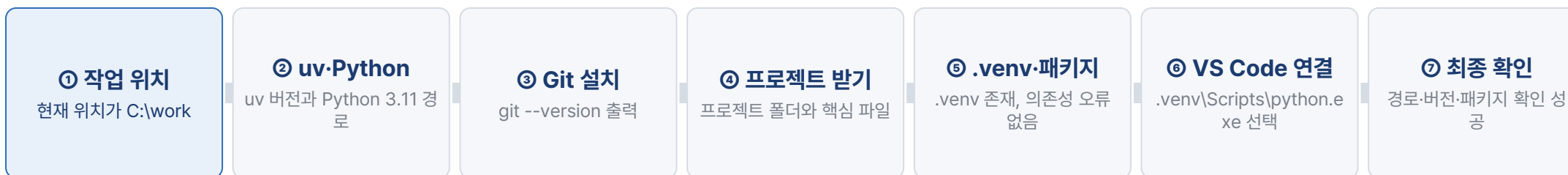
여섯 가지가 모두 확인되면 바로 1단계로 넘어갑니다

다음 → PART 1. 작업 위치를 고정합니다

PART 1

작업 위치를 먼저 고정합니다

왜: 긴 한글 경로에서 생기는 오류를 줄입니다 / 완료 증거: PowerShell의 현재 위치가 C:\work



다음 → PowerShell을 일반 권한으로 엽니다

PowerShell은 일반 권한으로 엽니다

① 작업 위치

② uv·Python

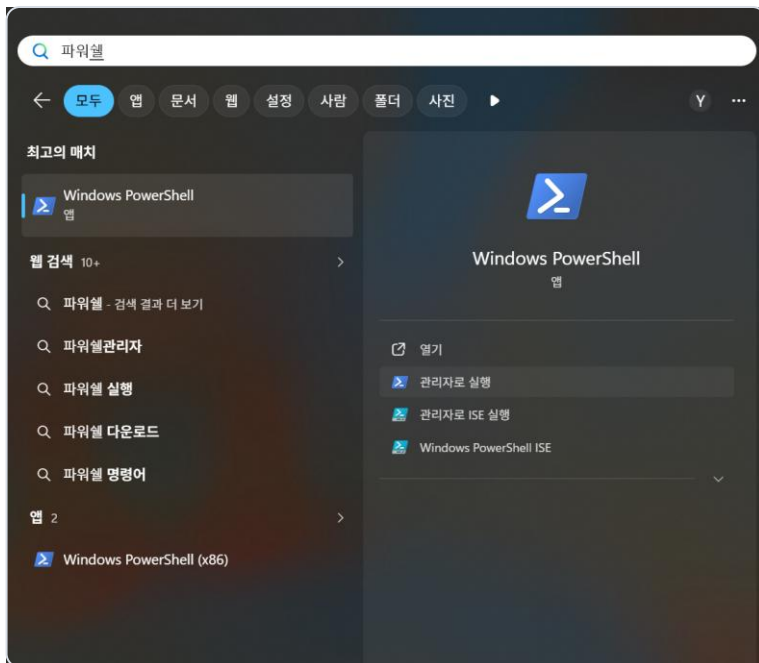
③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인



지금 할 일

- ① 작업 표시줄 검색창을 클릭합니다.
- ② Windows PowerShell 을 입력합니다.
- ③ 목록 맨 위 항목을 클릭해 엽니다.

관리자 권한으로 실행하지 않습니다.

설치가 권한 문제로 막힐 때만 교육 담당자에게 확인한 뒤 사용합니다.

정상 결과 ✓ 검은 명령 창이 열리고 커서가 깜박입니다

다르면 확인

검색 결과가 없으면 시작 메뉴에서 Windows PowerShell을 직접 찾습니다

다음 → C:\work 폴더를 만들고 이동합니다

C:\work가 보이면 작업 위치 준비가 끝납니다

① 작업 위치

② uv·Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

```
관리자: Windows PowerShell
PS C:\WINDOWS\system32> mkdir C:\work

디렉터리: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          2026-08-10   오전  8:12             work

PS C:\WINDOWS\system32> cd C:\work
```

① 아래 한 줄을 붙여넣고 Enter

`mkdir C:\work`

지금 할 일

- ① `mkdir C:\work` — 폴더를 만듭니다
- ② `cd C:\work` — 그 폴더로 이동합니다
- ③ 프롬프트가 `PS C:\work>` 로 바뀌었는지 봅니다

정상 결과 — 프롬프트가 이렇게 바뀝니다

`PS C:\work>`

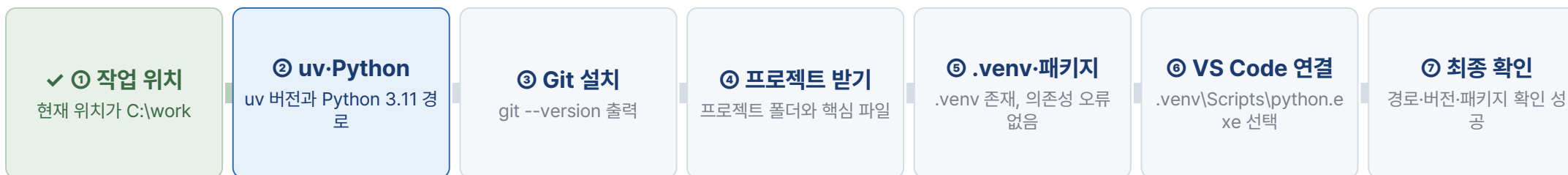
다르면 확인

이미 있습니다 라는 메시지가 나오면 폴더가 이미 있는 것이므로 ② 이동만 실행합니다

PART 2

uv가 Python 3.11을 준비합니다

왜: Python을 한 도구가 한 곳에서 관리하게 합니다 / 완료 증거: uv 버전과 Python 3.11 경로



다음 → 왜 uv에게 Python을 맡기는지 봅니다

Python은 uv가 한 곳에서 관리하게 합니다

두 방식을 함께 쓰면 어느 Python이 실행되는지 알 수 없게 됩니다.

이 교안이 사용하는 방법

uv를 설치합니다.

uv에게 Python 3.11을 준비하게 합니다.

프로젝트마다 .venv를 만듭니다.

→ 실행되는 Python이 항상 하나입니다.

직접 설치하는 사용하지 않습니다

Python을 따로 내려받아 설치하면

uv가 준비한 Python과 섞입니다.

조직 정책으로 uv가 Python을 받지 못할 때만

교육 담당자의 안내를 받아 사용합니다.

섞이면 이런 일이 생깁니다

설치 화면에는 3.11.9, 실행 결과에는 3.11.15처럼 서로 다른 값이 보여 어느 것이 실제로 쓰이는지 판단할 수 없습니다.

원칙

이 교안은 uv가 관리하는 Python 하나만 사용합니다

다음 → 설치 전에 주소와 정책을 확인합니다

설치 전에 주소와 정책을 먼저 확인합니다

[안전 확인](#)

✓ ① 작업 위치

② uv·Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

① 공식 주소인지 확인

명령 안의 주소가 astral.sh 로 시작하는지 눈으로 확인합니다.

② 스크립트를 먼저 봅니다

아래 명령으로 내용을 확인한 뒤 설치를 진행할 수 있습니다.

③ 조직 정책 확인

설치가 막히면 교육 담당자에게 승인된 방법을 확인합니다.

② 설치 스크립트를 먼저 읽어 보는 명령입니다.

```
powershell -c "irm https://astral.sh/uv/install.ps1 | more"
```

인터넷에서 받은 스크립트를 그대로 실행하는 명령입니다. 주소가 다르면 실행하지 않습니다.

원칙

막히면 관리자 권한으로 다시 시도하지 말고 승인된 지원 경로를 먼저 확인합니다

다음 → 공식 명령 한 줄로 uv를 설치합니다

공식 명령 한 줄로 uv를 설치합니다

✓ ① 작업 위치

② uv·Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

아래 한 줄을 복사해 PowerShell에 붙여넣고 Enter를 누릅니다.

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

지금 할 일

- ① 위 명령을 복사합니다.
- ② PowerShell에 붙여넣고 Enter를 누릅니다.
- ③ everything's installed! 문구가 나올 때까지 기다립니다.

정상 결과 — 마지막 줄이 이렇게 끝납니다

```
downloading uv ...  
installing to ...\.local\bin  
everything's installed!
```

정상 결과 ✓ everything's installed! 문구가 보이면 다음 쪽으로 이동합니다

다르면 확인

실행이 정책으로 막히면 `winget install --id=astral-sh.uv -e` 사용 가능 여부를 담당자에게 확인합니다

다음 → uv 버전이 보이는지 확인합니다

uv 버전이 보이면 설치가 끝난 것입니다

✓ ① 작업 위치

② uv-Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인

```
관리자: Windows PowerShell
PS C:\WINDOWS\system32> uv --version
uv 0.11.16 (135a36367 2026-05-21 x86_64-pc-windows-msvc)
PS C:\WINDOWS\system32>
```

설치가 끝나면 PowerShell을 닫고 새 창에서 실행합니다.

uv --version

정상

uv 뒤에 버전 번호가 보입니다.

명령을 찾을 수 없다고 나오면

- ① PowerShell 창을 완전히 닫습니다.
- ② 새 창을 열고 다시 실행합니다.
- ③ 그래도 없으면 13쪽으로 돌아가 설치를 다시 확인합니다.

정상 결과 ✓ uv 뒤에 버전 번호가 보이면 다음 쪽으로 이동합니다

다르면 확인

명령을 찾을 수 없다면 PowerShell을 완전히 닫고 새 창에서 다시 실행합니다

다음 → uv에게 Python 3.11을 준비하게 합니다

uv에게 Python 3.11을 준비하게 합니다

✓ ① 작업 위치

② uv·Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

아래 한 줄을 복사해 PowerShell에 붙여넣고 Enter를 누릅니다.

```
uv python install 3.11
```

지금 할 일

- ① 위 명령을 실행합니다.
- ② 내려받기가 끝날 때까지 기다립니다. 몇 분 걸릴 수 있습니다.
- ③ 설치 완료 문구를 확인합니다.

정상 결과 — 설치된 3.11 버전이 표시됩니다

Installed Python 3.11.x

정상 결과 ✓ Python 3.11 이 설치되었다는 문구가 보이면 다음 쪽으로 이동합니다

다르면 확인

다운로드가 막히면 11쪽의 대체 경로를 교육 담당자와 확인합니다

다음 → 준비된 Python의 경로를 확인합니다

같은 Python 경로가 보이는지 확인합니다

✓ ① 작업 위치

② uv-Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인

① 사용할 Python의 위치를 확인합니다.

```
uv python find 3.11
```

② 그 Python의 버전을 확인합니다.

```
uv run --python 3.11 python --version
```

두 가지가 모두 맞아야 다음으로 넘어갑니다

- 경로에 cpython-3.11 이 들어 있습니다.
- 버전이 Python 3.11.x 로 시작합니다. 이 값이 앞으로 계속 같아야 합니다.

정상 결과 ✓ 경로와 버전이 모두 3.11 을 가리키면 2단계가 끝난 것입니다

다르면 확인

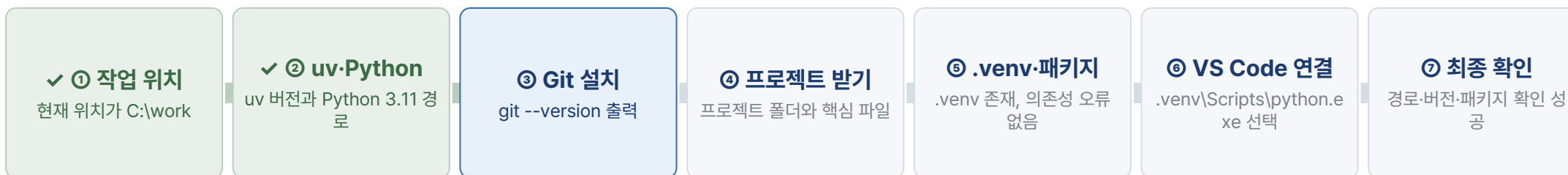
python 만 출력되고 버전이 없으면 Windows 설정 → 앱 → 앱 실행 별칭에서 python.exe 를 끕니다

다음 → PART 3. Git을 설치합니다

PART 3

Git으로 프로젝트를 받을 준비를 합니다

왜: 실습 파일을 빠짐없이 같은 구조로 받습니다 / 완료 증거: `git --version` 출력



다음 → 공식 Windows 설치 파일을 받습니다

공식 Windows x64 설치 파일을 받습니다

✓ ① 작업 위치

✓ ② uv-Python

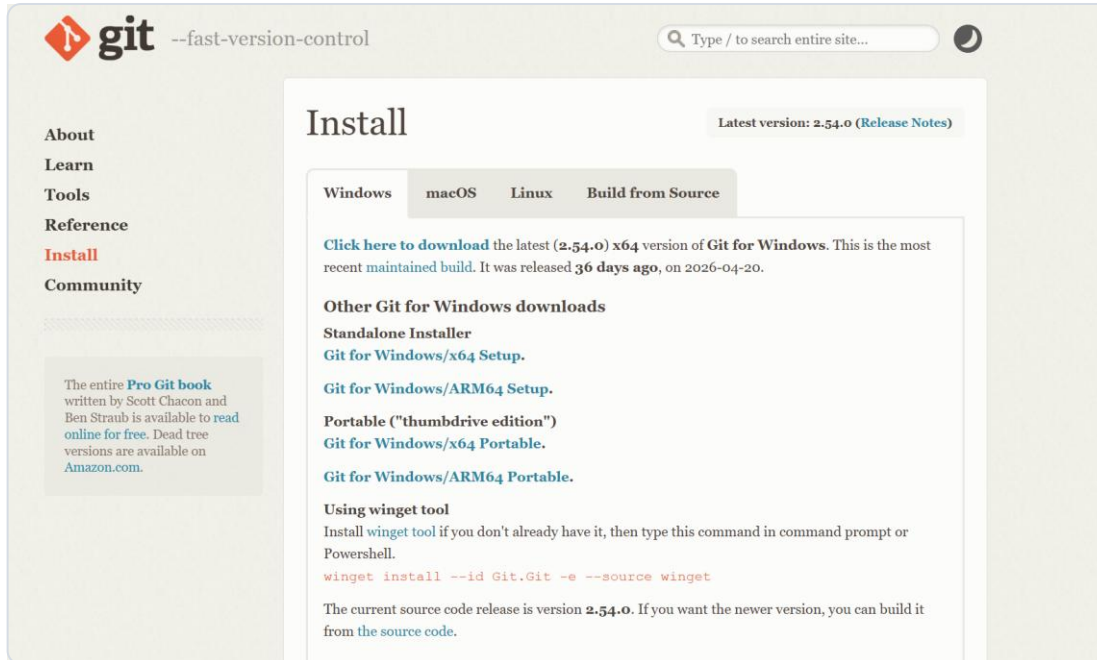
③ Git 설치

④ 프로젝트 받기

⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인



지금 할 일

① 브라우저에서 공식 주소를 엽니다.

`git-scm.com/install/windows`

② Windows 탭에서 64-bit Setup 을 선택합니다.

③ 내려받은 파일을 실행합니다.

특정 버전 번호가 아니라 현재 최신 유지 x64 설치 파일을 사용합니다.

교안 확인일: 2026-08-15

다르면 확인

ARM64 PC라면 x64 대신 ARM64 파일을 선택합니다. 확인이 어려우면 담당자에게 문의합니다

다음 → 설치를 시작하고 기본 경로를 유지합니다

설치를 시작하고 기본 경로를 그대로 둡니다

✓ ① 작업 위치

✓ ② uv-Python

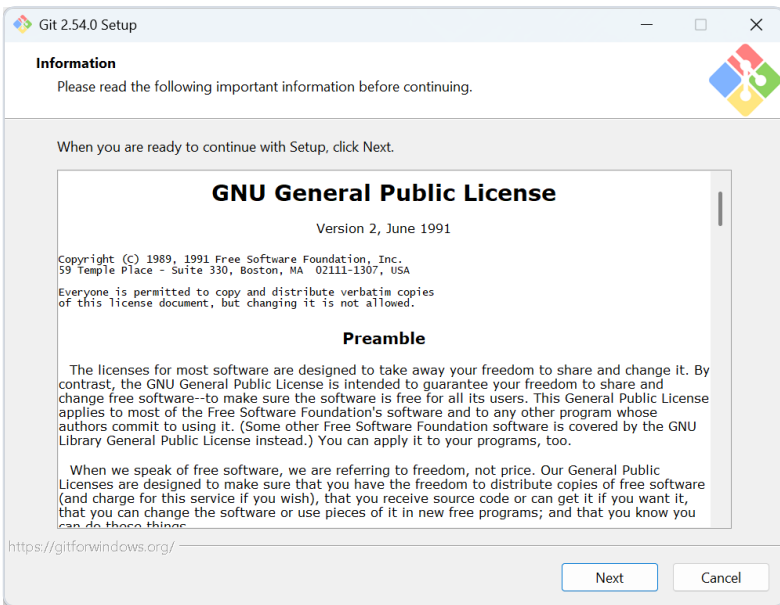
③ Git 설치

④ 프로젝트 받기

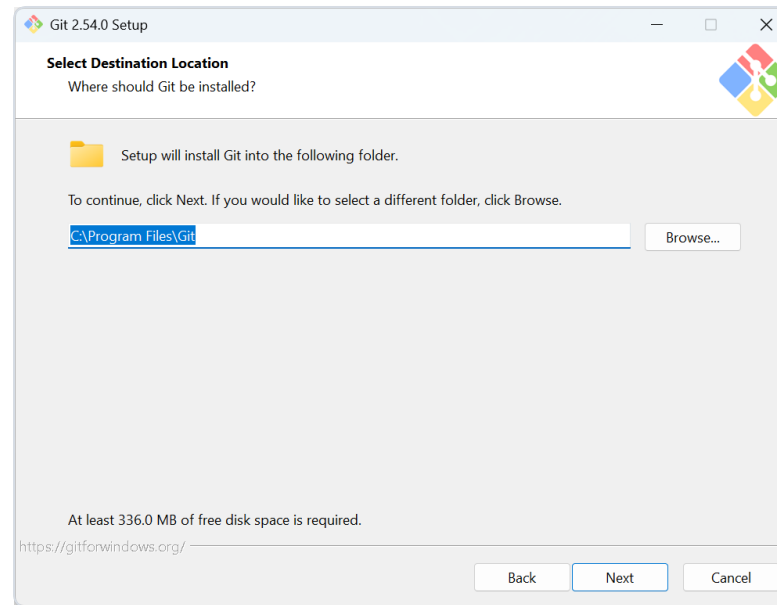
⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인



① 라이선스를 확인하고 Next



② 기본 경로를 그대로 두고 Next

다르면 확인

설치 경로를 바꾸지 않습니다. 다른 경로를 쓰면 이후 안내 화면과 달라집니다

다음 → 구성 요소와 시작 메뉴는 기본값을 사용합니다

구성 요소와 시작 메뉴는 기본값을 사용합니다

✓ ① 작업 위치

✓ ② uv-Python

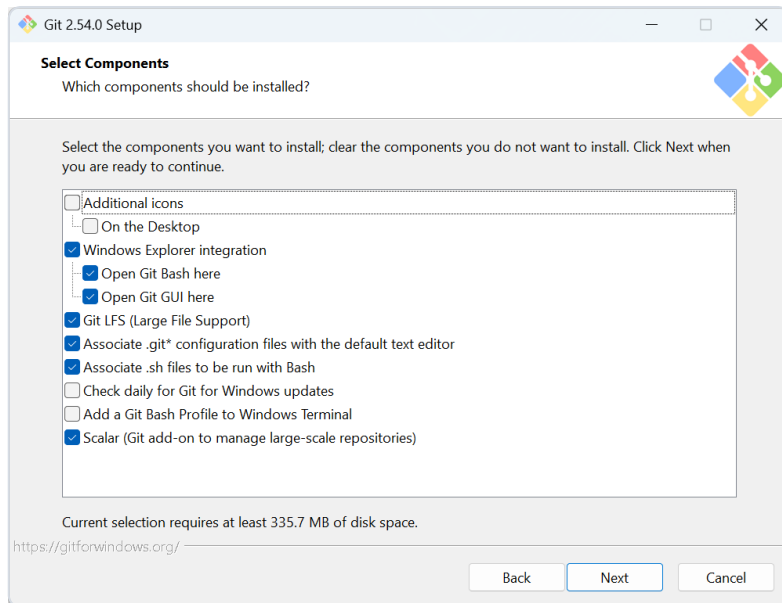
③ Git 설치

④ 프로젝트 받기

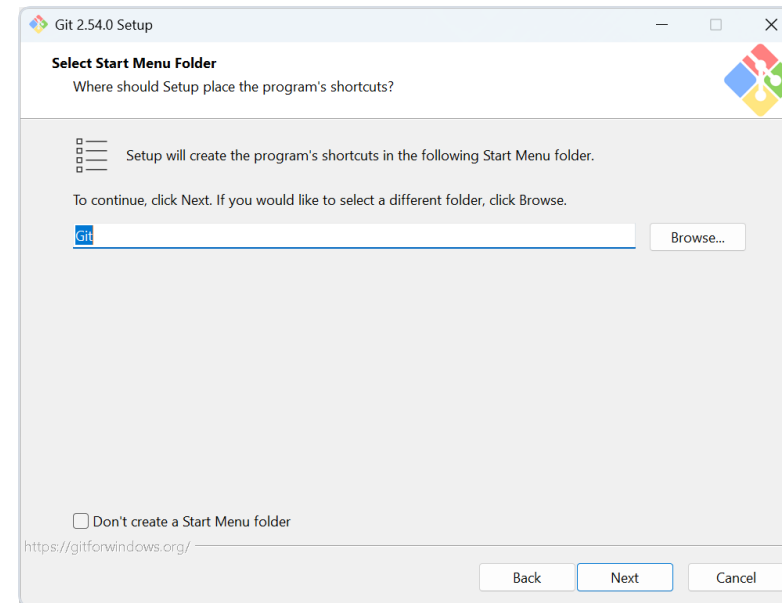
⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인



③ 체크를 바꾸지 않고 Next



④ 기본 이름을 그대로 두고 Next

원칙

이 두 화면에서는 바꿀 것이 없습니다. Next 만 누르면 됩니다

다음 → 편집기와 PATH만 직접 고릅니다

편집기와 PATH 두 가지만 직접 고릅니다

직접 선택

✓ ① 작업 위치

✓ ② uv-Python

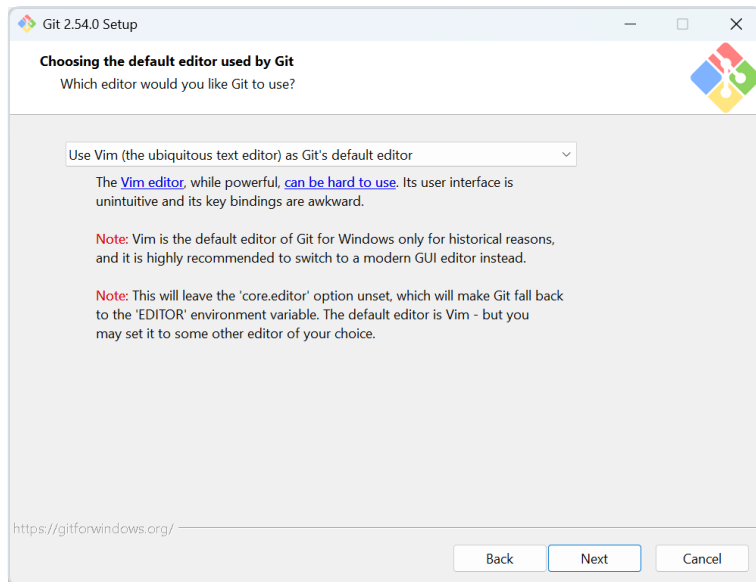
③ Git 설치

④ 프로젝트 받기

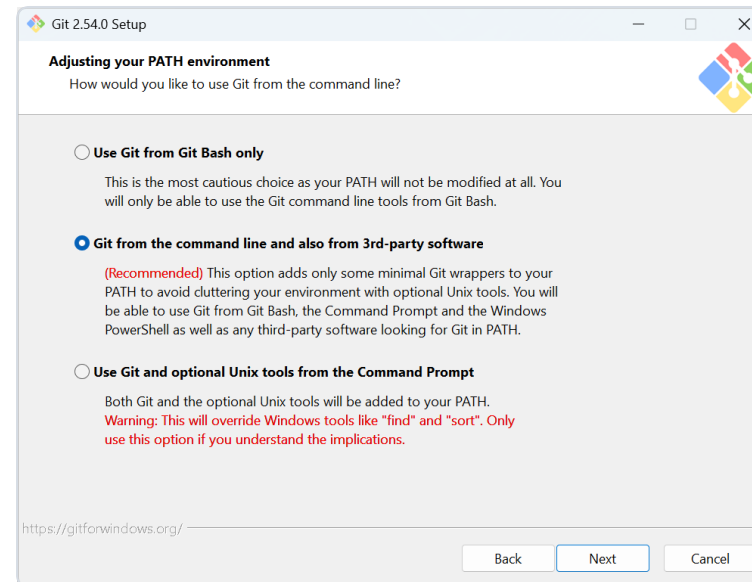
⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인



⑤ 편집기 = 목록에서 쉽게 답을 수 있는 항목(예: Notepad)



⑥ PATH = 가운데 권장 항목 → PowerShell에서 git 사용 가능

다르면 확인

PATH를 첫 번째 항목으로 고르면 PowerShell에서 git 을 찾지 못합니다

다음 → 나머지 설정은 기본값 표로 한 번에 확인합니다

나머지 설정은 모두 기본값으로 둡니다

✓ ① 작업 위치	✓ ② uv-Python	③ Git 설치	④ 프로젝트 받기	⑤ .venv-패키지	⑥ VS Code 연결	⑦ 최종 확인
HTTPS 인증서	기본값 유지	조직 안내가 있으면 그 값을 먼저 따릅니다				
SSH 실행 파일	기본값 유지	이번 실습은 HTTPS로 받으므로 영향이 없습니다				
줄바꿈 처리	기본값 유지	이번 실습 결과에 영향이 없습니다				
터미널 종류	기본값 유지	이 과정은 PowerShell을 사용합니다				
git pull 기본 동작	기본값 유지	받기만 하므로 판단이 필요 없습니다				
자격 증명 관리자	기본값 유지	공개 저장소는 로그인이 필요 없습니다				
Enable symbolic links	선택하지 않음	권한 문제가 생길 수 있어 기본 상태로 둡니다				

원칙

조직의 보안·인증서 안내가 있으면 그 안내를 먼저 따릅니다

다음 → 설치를 끝내고 새 PowerShell을 엽니다

설치를 끝내고 새 PowerShell을 엽니다

✓ ① 작업 위치

✓ ② uv·Python

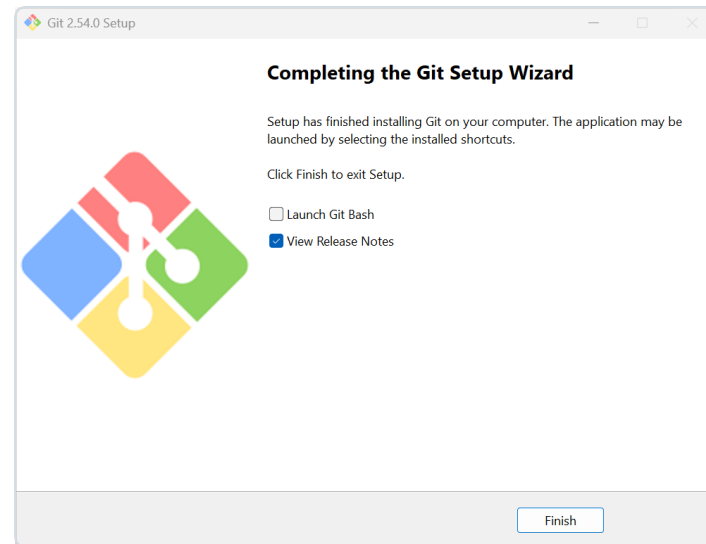
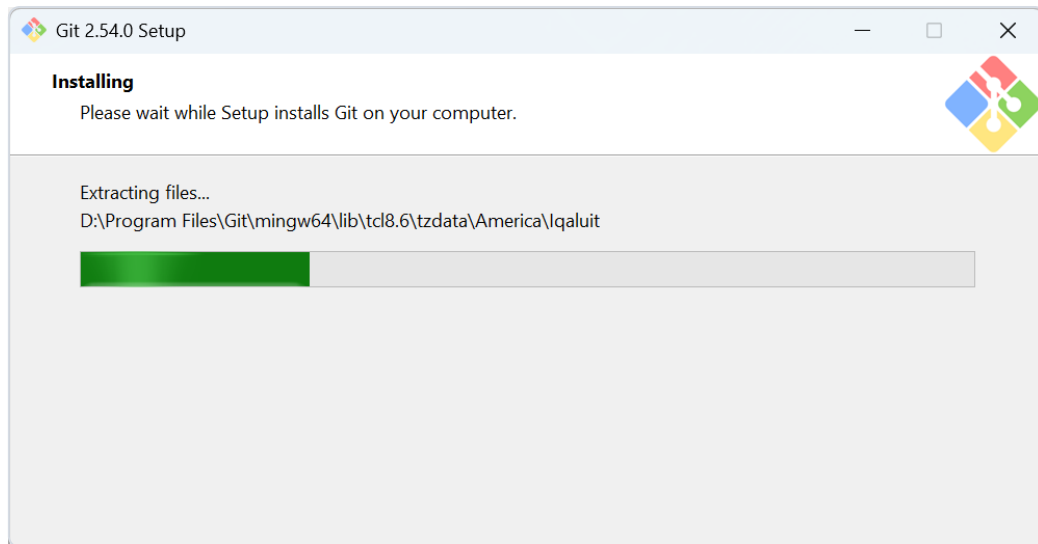
③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인



⑦ Install → ⑧ 진행 완료 → ⑨ Finish → ⑩ 기존 PowerShell 창을 닫고 새 창을 엽니다

다르면 확인

기존 창을 그대로 쓰면 git 명령을 찾지 못합니다. 반드시 새 창을 엽니다

다음 → git 버전이 보이는지 확인합니다

git 버전이 보이면 프로젝트를 받을 수 있습니다

✓ ① 작업 위치

✓ ② uv·Python

③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

```
관리자: Windows PowerShell
PS C:\WINDOWS\system32> git --version
git version 2.54.0.windows.1
PS C:\WINDOWS\system32>
```

새 PowerShell 창에서 실행합니다.

git --version

여기까지 완료

- ✓ 현재 위치가 C:\work
- ✓ uv 버전과 Python 3.11 경로
- ✓ git --version 출력

세 칸이 초록으로 바뀌었습니다.

✓ ① 작업 위치

현재 위치가 C:\work

✓ ② uv·Python

uv 버전과 Python 3.11 경로

✓ ③ Git 설치

git --version 출력

④ 프로젝트 받기

프로젝트 폴더와 핵심 파일

⑤ .venv·패키지

.venv 존재, 의존성 오류 없음

⑥ VS Code 연결

.venv\Scripts\python.exe 선택

⑦ 최종 확인

경로·버전·패키지 확인 성공

다르면 확인

명령을 찾을 수 없다면 ① 새 창 → ② 21쪽 PATH 선택 → ③ 재설치 순서로 확인합니다

다음 → PART 4. 승인된 저장소를 받습니다

PART 4

승인된 저장소를 작업 폴더에 복사합니다

왜: 실습 파일을 모두 같은 구조로 받습니다 / 완료 증거: 프로젝트 폴더와 핵심 파일



다음 → 주소와 현재 위치를 먼저 확인합니다

받기 전에 주소와 현재 위치를 확인합니다

[사전 확인](#)

✓ ① 작업 위치

✓ ② uv-Python

✓ ③ Git 설치

④ 프로젝트 받기

⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인

① 저장소 주소를 확인합니다

`github.com/kaistaieducation/ai-advanced-03`

교육 담당자가 안내한 주소와 같은지 눈으로 확인합니다.

② 현재 위치를 확인합니다

프롬프트가 `PS C:\work>` 인지 봅니다.

다르면 `cd C:\work` 를 먼저 실행합니다.

왜 위치가 중요한가요

명령을 실행한 폴더 아래에 프로젝트가 만들어집니다. `C:\work` 가 아닌 곳에서 실행하면 이후 모든 안내의 경로가 달라집니다.

원칙

안내받은 주소와 다르면 실행하지 않고 교육 담당자에게 확인합니다

다음 → `git clone` 으로 프로젝트를 복사합니다

git clone 으로 프로젝트를 복사합니다

✓ ① 작업 위치

✓ ② uv-Python

✓ ③ Git 설치

④ 프로젝트 받기

⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인

아래 한 줄을 복사해 PowerShell에 붙여넣고 Enter를 누릅니다.

```
git clone https://github.com/kaistaieducation/ai-advanced-03.git
```

```
PS C:\WINDOWS\system32> cd C:\work
PS C:\work> git clone https://github.com/kaistaieducation/ai-advanced-03.git
Cloning into 'ai-advanced-03'...
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 16 (delta 0), reused 16 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (16/16), 3.40 MiB | 16.43 MiB/s, done.
PS C:\work>
```

지금 할 일

- ① 위 명령을 붙여넣고 Enter를 누릅니다.
- ② 내려받기가 끝날 때까지 기다립니다.
- ③ 마지막 줄이 done. 으로 끝나는지 봅니다.

정상 결과 ✓ 마지막 줄이 done. 으로 끝나면 다음 쪽으로 이동합니다

다르면 확인

주소 오류·인터넷 문제·이미 있는 폴더는 29쪽에서 구분합니다

다음 → 폴더와 핵심 파일이 생겼는지 봅니다

폴더와 핵심 파일이 보이면 복사가 끝난 것입니다

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

① 프로젝트 폴더로 이동합니다.

```
cd C:\work\ai-advanced-03
```

② 폴더 안의 파일을 봅니다.

```
dir
```

정상 결과 — 다음 항목이 보입니다

```
README.md
requirements.txt
day01 day02 day03 day04
```

✓ ① 작업 위치

현재 위치가 C:\work

✓ ② uv·Python

uv 버전과 Python 3.11 경
로

✓ ③ Git 설치

git --version 출력

✓ ④ 프로젝트 받기

프로젝트 폴더와 핵심 파일

⑤ .venv·패키지

.venv 존재, 의존성 오류
없음

⑥ VS Code 연결

.venv\Scripts\python.e
xe 선택

⑦ 최종 확인

경로·버전·패키지 확인 성
공

다르면 확인

폴더가 없으면 27쪽으로, 파일이 비어 있으면 29쪽으로 돌아갑니다

복사가 안 되면 세 가지를 확인합니다

[문제 해결](#)

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

주소를 찾을 수 없습니다

repository not found 또는 404

- 주소의 철자를 다시 확인합니다.
- 26쪽으로 돌아갑니다.

연결이 되지 않습니다

could not resolve host / timeout

- 인터넷 연결을 확인합니다.
- 조직 프록시가 있으면 담당자에게 확인합니다.

폴더가 이미 있습니다

destination path already exists

- 이미 받은 것입니다.
- cd 로 들어가 28쪽 확인만 합니다.

세 가지에 해당하지 않으면 46쪽 문제 해결 색인에서 증상을 찾습니다.

원칙

증상을 먼저 고르고, 표시된 쪽으로 돌아가 그 단계만 다시 합니다

PART 5

프로젝트 전용 Python 공간을 만듭니다

왜: 프로젝트마다 필요한 기능을 섞이지 않게 나눕니다 / 완료 증거: .venv 존재와 의존성 오류 없음



다음 → .venv 가 무엇인지 한 문장으로 봅니다

.venv 는 이 프로젝트만 쓰는 상자입니다

.venv = 이 프로젝트만 사용하는 Python 공간이라는 뜻입니다.

공용 공간 하나만 쓰면

모든 프로젝트가 같은 자리에 기능을 설치합니다.
한 실습이 요구하는 버전이 다른 실습을 망칠 수 있습니다.



프로젝트마다 .venv 를 두면

이 프로젝트가 쓰는 기능만 이 안에 설치됩니다.
다른 실습을 건드리지 않습니다.

기억할 한 줄

.venv 는 프로젝트 폴더 안에 생기는 폴더입니다. 지우면 다시 만들면 됩니다.

핵심

프로젝트마다 상자를 따로 두면 다른 실습의 기능과 섞이지 않습니다

다음 → 명령을 실행할 폴더로 이동합니다

명령을 실행하기 전에 프로젝트 폴더로 이동합니다

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

아래 한 줄을 복사해 PowerShell에 붙여넣고 Enter를 누릅니다.

```
cd C:\work\ai-advanced-03
```

잘못된 위치

```
PS C:\work>
```

여기서 실행하면 .venv 가 엉뚱한 곳에 생깁니다.

올바른 위치

```
PS C:\work\ai-advanced-03>
```

여기서 실행해야 프로젝트 안에 생깁니다.

정상 결과 ✓ 프롬프트 끝이 ai-advanced-03> 이면 다음 쪽으로 이동합니다

다르면 확인

프롬프트가 PS C:\work\ai-advanced-03> 가 아니면 아래 명령을 다시 실행합니다

다음 → Python 3.11로 .venv 를 만듭니다

Python 3.11로 .venv 를 만듭니다

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

아래 한 줄을 복사해 PowerShell에 붙여넣고 Enter를 누릅니다.

```
uv venv --python 3.11
```

지금 할 일

- ① 위 명령을 실행합니다.
- ② 사용된 Python 버전 줄을 확인합니다.
- ③ 만들어진 위치가 .venv 인지 확인합니다.

정상 결과 — 두 줄이 보입니다

```
Using CPython 3.11.x  
Creating virtual environment at: .venv
```

정상 결과 ✓ 버전이 3.11 이고 위치가 .venv 이면 다음 쪽으로 이동합니다

다르면 확인

32쪽 위치를 먼저 확인하고, Python을 찾지 못하면 15쪽으로 돌아갑니다

다음 → 실습에 필요한 기능을 설치합니다

실습에 필요한 기능을 .venv 에 설치합니다

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

아래 한 줄을 복사해 PowerShell에 붙여넣고 Enter를 누릅니다.

```
uv pip install -r requirements.txt
```

requirements.txt 가 무엇인가요

실습 코드가 사용하는 기능의 목록입니다.

이 프로젝트에는 다음이 들어 있습니다.

tensorflow · keras · numpy · pandas · matplotlib · scikit-learn

기다리는 동안

- 내려받을 파일이 커서 몇 분 이상 걸릴 수 있습니다.
- 화면이 멈춘 것처럼 보여도 창을 닫지 않습니다.
- 마지막 줄이 나올 때까지 기다립니다.

다르면 확인

설치가 오래 걸리는 것은 정상입니다. 오류로 끝나면 35쪽에서 증상을 고릅니다

다음 → 설치가 제대로 끝났는지 검사합니다

의존성 오류가 없으면 프로젝트 환경이 준비된 것입니다

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

설치가 끝나면 검사 명령을 실행합니다.

`uv pip check`

정상 결과

All installed packages are compatible

네트워크 오류

연결이 끊긴 것입니다. 34쪽 명령을 다시 실행합니다.

빌드 오류

설치 중 컴파일 오류가 보이면 교육 담당자에게 확인합니다.

위치 오류

requirements.txt 를 찾지 못하면 32쪽으로 돌아갑니다.

✓ ① 작업 위치

현재 위치가 C:\work

✓ ② uv·Python

uv 버전과 Python 3.11 경로

✓ ③ Git 설치

git --version 출력

✓ ④ 프로젝트 받기

프로젝트 폴더와 핵심 파일

✓ ⑤ .venv·패키지

.venv 존재, 의존성 오류 없음

⑥ VS Code 연결

.venv\Scripts\python.exe 선택

⑦ 최종 확인

경로·버전·패키지 확인 성공

다르면 확인

네트워크 오류는 다시 실행, 빌드 오류는 담당자 확인, 위치 오류는 32쪽으로 돌아갑니다

PART 6

VS Code를 프로젝트 환경에 연결합니다

왜: 파일과 터미널과 Python을 한 화면에서 씁니다 / 완료 증거: `.venv\Scripts\python.exe` 선택



다음 → 공식 설치 파일을 받아 설치합니다

공식 Windows x64 설치 파일을 받아 시작합니다

✓ ① 작업 위치

✓ ② uv-Python

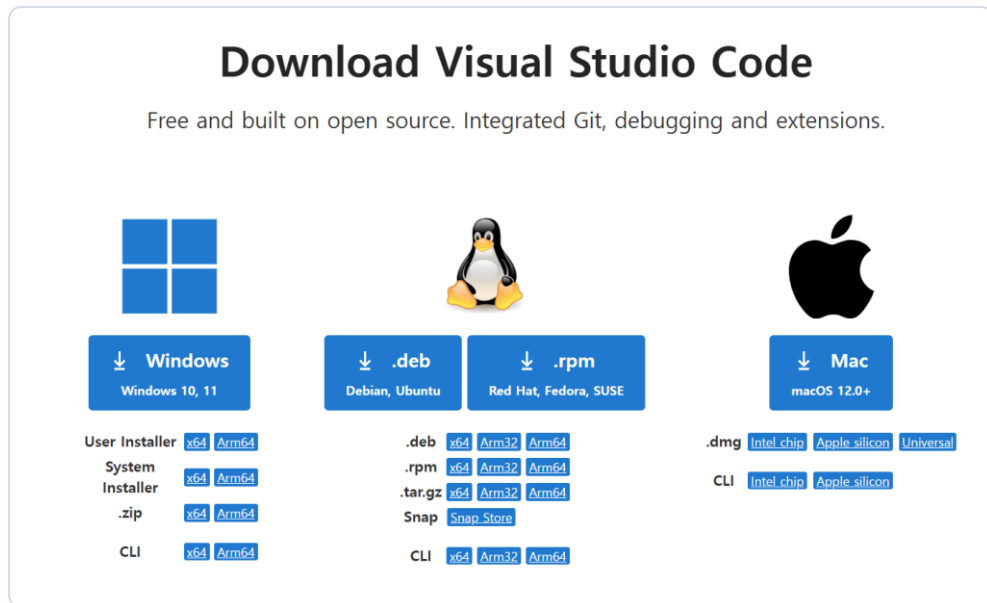
✓ ③ Git 설치

✓ ④ 프로젝트 받기

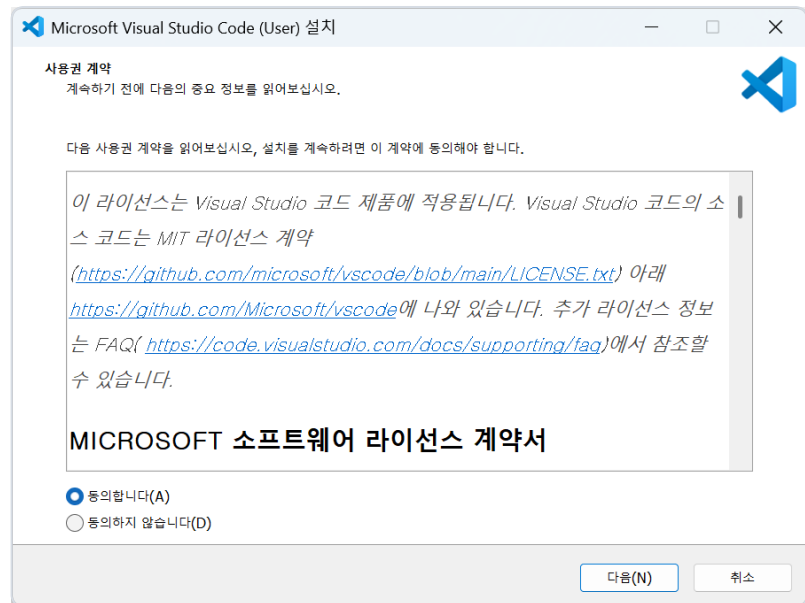
✓ ⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인



① Windows x64 설치 파일 받기



② 동의 선택 → 다음

다르면 확인

설치 경로와 시작 메뉴 이름은 바꾸지 않습니다

다음 → 필요한 항목만 체크하고 설치를 끝냅니다

필요한 항목만 체크하고 설치를 끝냅니다

직접 선택

✓ ① 작업 위치

✓ ② uv-Python

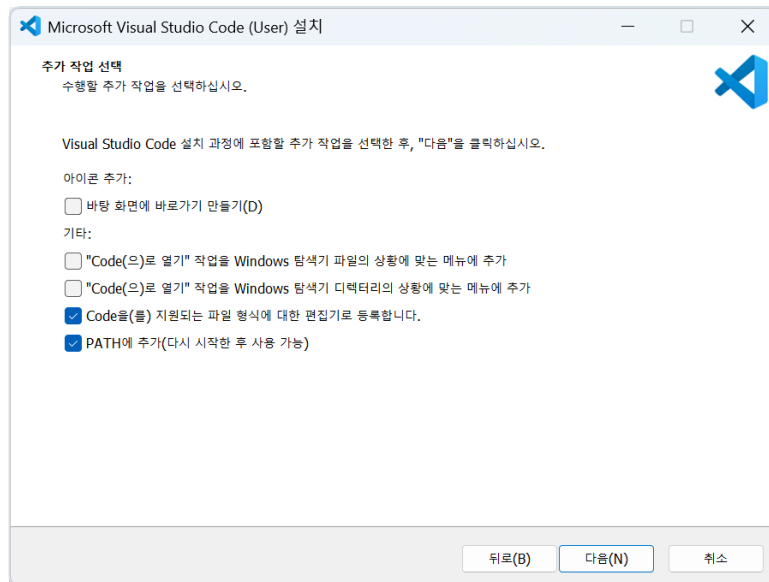
✓ ③ Git 설치

✓ ④ 프로젝트 받기

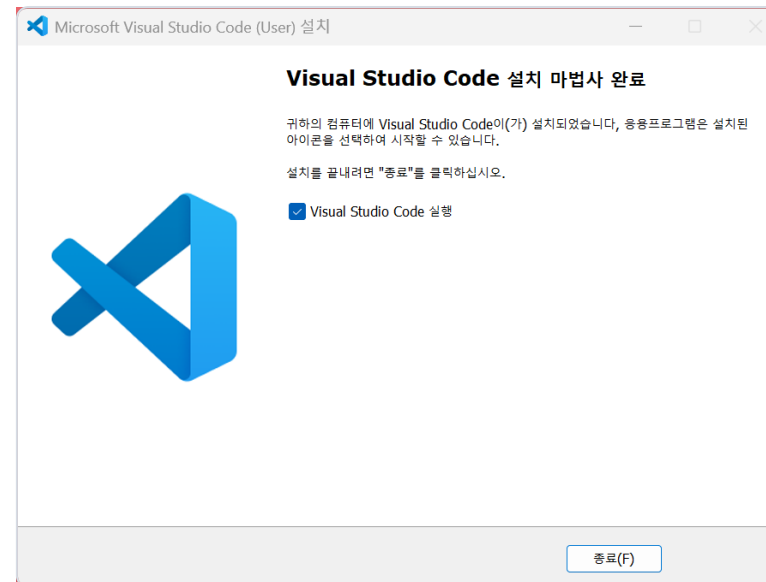
✓ ⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인



③ PATH에 추가 가 켜져 있는지 확인 → 설치



④ 설치 완료 → 마침 → VS Code 실행

다르면 확인

PATH 항목을 끄면 터미널에서 code 명령을 쓸 수 없습니다. 켜진 상태로 둡니다

다음 → Microsoft Python 확장을 설치합니다

Microsoft Python 확장을 먼저 설치합니다

필수

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

✓ ⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

지금 할 일

- ① 왼쪽 세로 막대에서 Extensions 를 클릭합니다.
- ② 검색창에 Python 을 입력합니다.
- ③ **게시자가 Microsoft 인 항목을 고릅니다.**
- ④ Install 을 누릅니다.

정상 결과

Install 단추가 Installed 로 바뀝니다
확장 이름 옆에 사용 중 표시가 보입니다

왜 이 단계를 건너뛰면 안 되나요

Python: Select Interpreter 명령은 이 확장이 제공합니다. 확장이 없으면 다음 단계가 진행되지 않습니다.

다르면 확인

이 확장이 없으면 42쪽의 Python: Select Interpreter 명령이 보이지 않습니다

다음 → 프로젝트 폴더 하나만 엽니다

프로젝트 폴더 하나만 엽니다

✓ ① 작업 위치

✓ ② uv·Python

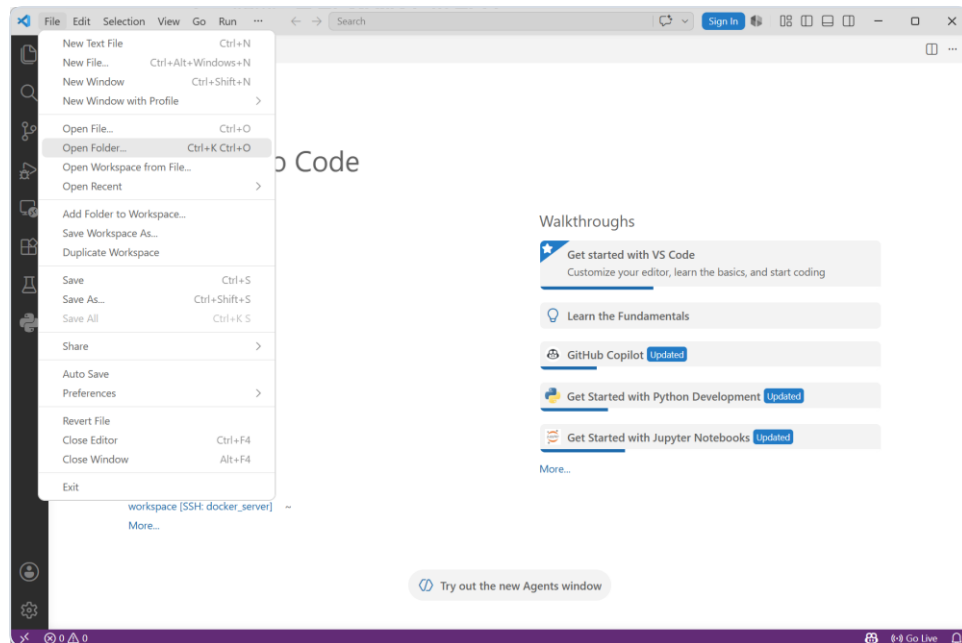
✓ ③ Git 설치

✓ ④ 프로젝트 받기

✓ ⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인



지금 할 일

- ① File 메뉴를 엽니다.
- ② Open Folder... 를 선택합니다.
- ③ 아래 폴더를 고릅니다.

C:\work\ai-advanced-03

폴더를 열면 왼쪽에 파일 목록이 나타납니다.

정상 결과 ✓ 왼쪽 탐색기에 ai-advanced-03 과 그 안의 파일이 보이면 정상입니다

다르면 확인

파일 하나만 열면 .venv 를 찾지 못합니다. 반드시 폴더를 엽니다

다음 → 출처를 확인한 뒤 신뢰 여부를 정합니다

출처를 확인한 뒤 신뢰 여부를 정합니다

안전 판단

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

✓ ⑤ .venv·패키지

⑥ VS Code 연결

⑦ 최종 확인

신뢰합니다 — 이럴 때만

- 26쪽에서 확인한 주소에서 직접 받은 폴더입니다.
- 폴더 경로가 C:\work\ai-advanced-03 입니다.
- 예, 작성자를 신뢰합니다 를 선택합니다.

신뢰하지 않습니다 — 이럴 때

- 어디서 받았는지 확실하지 않습니다.
- 경로나 폴더 이름이 안내와 다릅니다.
- 제한 모드를 유지하고 담당자에게 확인합니다.

Workspace Trust 가 무엇인가요

처음 여는 폴더의 코드를 자동으로 실행해도 되는지 묻는 화면입니다. 창이 나타나지 않으면 그대로 다음 쪽으로 넘어갑니다.

원칙

무조건 신뢰하지 않습니다. 안내받은 주소에서 받은 폴더일 때만 신뢰합니다

다음 → 프로젝트의 .venv Python을 선택합니다

프로젝트의 .venv Python을 선택합니다

✓ ① 작업 위치

✓ ② uv-Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

✓ ⑤ .venv-패키지

⑥ VS Code 연결

⑦ 최종 확인

① 명령 팔레트를 열고 이 명령을 고릅니다.

Ctrl + Shift + P → Python: Select Interpreter

② 목록에서 이 경로를 고릅니다.

.venv\Scripts\python.exe

고를 항목을 구분하는 법

- 경로가 .venv 로 시작하는 항목을 고릅니다.
- 전역(Global) 표시가 붙은 항목은 고르지 않습니다.

정상 결과

오른쪽 아래 상태 표시줄에
.venv 를 사용하는 Python 3.11.x 가 보입니다

✓ ① 작업 위치

현재 위치가 C:\work

✓ ② uv-Python

uv 버전과 Python 3.11 경
로

✓ ③ Git 설치

git --version 출력

✓ ④ 프로젝트 받기

프로젝트 폴더와 핵심 파일

✓ ⑤ .venv-패키지

.venv 존재, 의존성 오류
없음

✓ ⑥ VS Code 연결

.venv\Scripts\python.e
xe 선택

⑦ 최종 확인

경로·버전·패키지 확인 성
공

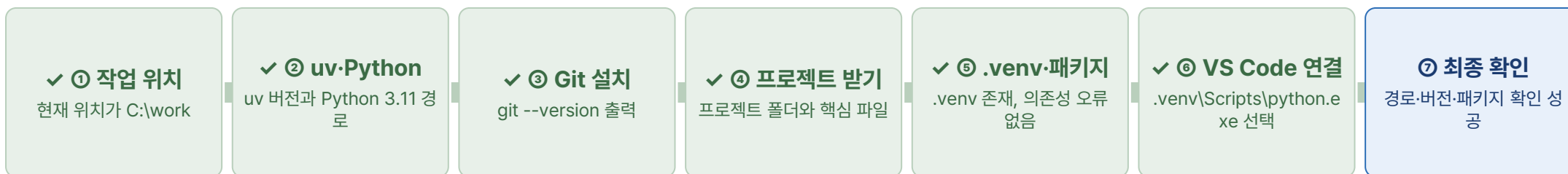
다르면 확인

목록에 없으면 새로 고침 후 Enter interpreter path 로 직접 경로를 지정합니다

PART 7

실제로 같은 Python이 실행되는지 확인합니다

왜: 선택만으로는 끝난 것이 아닙니다 / 완료 증거: 경로·버전·패키지 확인이 모두 성공



다음 → 새 터미널에서 경로와 버전을 확인합니다

새 터미널에서 Python 경로와 버전을 확인합니다

✓ ① 작업 위치

✓ ② uv-Python

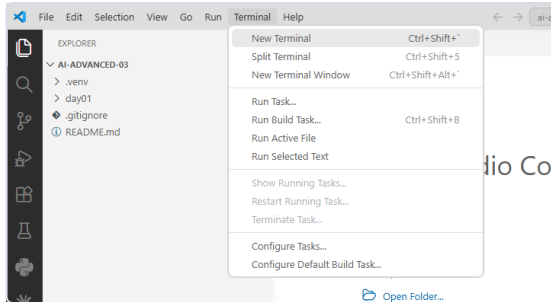
✓ ③ Git 설치

✓ ④ 프로젝트 받기

✓ ⑤ .venv-패키지

✓ ⑥ VS Code 연결

⑦ 최종 확인



① 새 터미널에 아래 한 줄을 붙여넣고 Enter를 누릅니다.

```
python -c "import sys; print(sys.executable); print(sys.version)"
```

정상 결과 — 두 줄이 이렇게 나옵니다

```
C:\work\ai-advanced-03\.venv\Scripts\python.exe
3.11.x ...
```

정상 결과 ✓ 경로에 .venv 가 있고 버전이 3.11 이면 마지막 쪽으로 이동합니다

다르면 확인

경로에 .venv 가 없으면 42쪽으로 돌아가 인터프리터를 다시 고릅니다

다음 → 패키지가 실제로 동작하는지 확인합니다

패키지가 동작하면 모든 준비가 끝났습니다

최종 확인

✓ ① 작업 위치

✓ ② uv·Python

✓ ③ Git 설치

✓ ④ 프로젝트 받기

✓ ⑤ .venv·패키지

✓ ⑥ VS Code 연결

⑦ 최종 확인

① 의존성 검사를 실행합니다.

```
uv pip check
```

② 실습에 쓰는 기능이 실제로 불러와지는지 확인합니다.

```
python -c "import tensorflow, keras, numpy, pandas, matplotlib, sklearn; print('OK')"
```

정상 결과 — 두 줄이 모두 나오면 완료입니다

```
All installed packages are compatible  
OK
```

다르면 확인

import 오류가 나면 34~35쪽으로 돌아가 설치를 다시 확인합니다

다음 → 막혔던 곳이 있다면 문제 해결 색인으로

증상으로 찾으면 돌아갈 쪽이 보입니다

[문제 해결](#)

막혔을 때 처음부터 다시 하지 말고 아래에서 증상을 먼저 고릅니다.

증상	가장 먼저 확인할 것	돌아갈 쪽
명령을 찾을 수 없습니다	새 창을 열지 않았거나 PATH 설정	14쪽 · 21쪽 · 24쪽
clone 이 실패합니다	주소·인터넷·이미 있는 폴더	29쪽
.venv 가 만들어지지 않습니다	실행 위치가 프로젝트 폴더가 아님	32쪽 · 33쪽
패키지 설치가 실패합니다	네트워크 끊김 또는 빌드 오류	35쪽
인터프리터 목록이 없습니다	Python 확장이 설치되지 않음	39쪽
선택했는데 경로가 다릅니다	전역 Python을 고름	42쪽
import 가 실패합니다	패키지 설치가 끝나지 않음	34쪽 · 35쪽
Python 만 출력됩니다	Windows 앱 실행 별칭	16쪽 · Windows 설정

원칙

한 번에 한 단계만 다시 합니다. 그 단계의 정상 결과가 나오면 다시 앞으로 진행합니다

다음 → 일곱 개의 완료 증거를 직접 확인합니다

일곱 개의 완료 증거를 직접 확인합니다

체크리스트

하나라도 비어 있으면 그 단계로 돌아가 그 부분만 다시 합니다.

☐	① 작업 위치	현재 위치가 C:\work	9쪽
☐	② uv·Python	uv 버전과 Python 3.11 경로	16쪽
☐	③ Git 설치	git --version 출력	24쪽
☐	④ 프로젝트 받기	프로젝트 폴더와 핵심 파일	28쪽
☐	⑤ .venv·패키지	.venv 존재, 의존성 오류 없음	35쪽
☐	⑥ VS Code 연결	.venv\Scripts\python.exe 선택	42쪽
☐	⑦ 최종 확인	경로·버전·패키지 확인 성공	45쪽

인쇄해서 옆에 두고 하나씩 표시하며 확인해도 됩니다.

완료 판정

일곱 칸이 모두 채워지면 개발 환경 구성이 끝난 것입니다

다음 → 일곱 단계를 직접 연결해 봅니다

일곱 단계를 순서대로 놓아 봅시다

활동

아래 카드를 순서대로 배열하고 각 단계의 완료 증거를 연결해 보세요.

.venv·패키지 준비	.venv 존재, 의존성 오류 없음	Git 설치	git --version 출력
최종 실행 확인	경로·버전·패키지 확인 성공	작업 위치 준비	현재 위치가 C:\work
VS Code 연결	.venv\Scripts\python.exe 선택	uv·Python 준비	uv 버전과 Python 3.11 경로
	프로젝트 받기	프로젝트 폴더와 핵심 파일	

왼쪽 카드를 순서대로 놓고, 오른쪽 완료 증거와 짝을 지어 보세요.

생각할 기준

무엇이 끝나야 다음 일을 할 수 있는지 따라가면 순서가 보입니다

다음 → 정답과 완성 지도

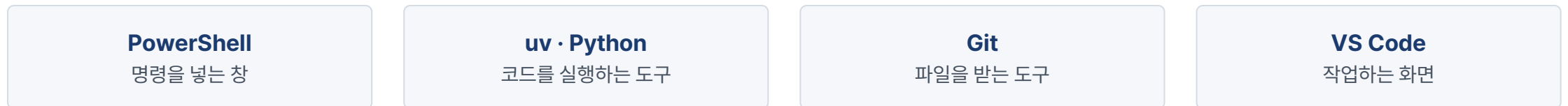
개발 환경 구성은 하나의 연결 흐름입니다

정답

4쪽에서 본 것과 같은 순서·같은 색·같은 이름입니다.



일곱 칸이 모두 초록이면, 실습 코드를 실행할 준비가 끝난 것입니다.



정답

일곱 단계는 앞 단계의 완료 증거가 있어야 다음 단계로 넘어갑니다

다음 → 이제 첫 실습 파일을 엽니다

이제 첫 실습 파일을 열어 확인합니다

준비가 끝났으므로 다음 실습으로 바로 이어집니다.

① 폴더 열기

VS Code에서 C:\work\ai-advanced-03 가 열려 있는지 확인합니다



② 실습 파일 열기

day01 폴더의 실습 파일을 엽니다



③ 실행 확인

오른쪽 아래 Python이 .venv 인지 확인한 뒤 실행합니다

환경이 흔들리면

46쪽 문제 해결 색인에서 증상을 찾고, 47쪽 체크리스트로 일곱 개 완료 증거를 다시 확인합니다.

마무리

환경이 준비되면 실습 파일을 열고 실행 결과를 확인하는 것으로 시작합니다